EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**A hash-table function based on adding character codes is suitable for protecting real passwords.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.





DO NOW — RECALL

# C# Login Project · Code a Hash Table

Name: _____ Class: _____ Date: _____

**1. A login check uses validUser && validPassword . What is the result when validUser is true and validPassword is false? Explain.**

**2. A D-type flip-flop currently stores Q = 1. D changes to 0 but there is no triggering clock edge. What is Q?**

**3. A queue receives A, B and C in that order. Remove one item, then enqueue D. List the remaining items from front to rear.**

**4. A five-bucket hash table uses h(k) = k MOD 5 with linear probing. Insert keys 7, 12 and 17 in that order into an empty table. State each final index.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

False. AND ( && ) is true only when both conditions are true.

### 2. Model answer

Q remains 1. The input is sampled only when the triggering clock edge occurs.

### 3. Model answer

B, C, D. A leaves first because a queue is FIFO.

### 4. Model answer

7 hashes to 2 and is stored at 2. 12 also hashes to 2, so probes to 3. 17 also hashes to 2; 2 and 3 are occupied, so it goes to 4.

### By the end, you can…

• Write a simple hash function and store usernames in an array.

• Resolve collisions using linear probing and search for usernames.

• Write synoptic MCQs on linked structures and hash tables.

### Vocabulary — know these terms

hash table — A table using a hash function to locate data from a key.

hash function — A repeatable mapping from key to bucket index.

bucket — A location in the hash table.

collision — Two keys map to the same bucket.

linear probing — Resolve collisions by checking successive buckets.

key — The value used to look up an entry.

lookup — Find a stored item using its key.

load factor — Fraction of table buckets occupied.





YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

CODING 3 · LOOKUP
Write a search that follows the same probing sequence. Return found/not found, including for a username never inserted.

### Question 4

SYNOPTIC MCQ BANK
Write MCQs on linked lists, directed/undirected graphs, stacks, queues, trees, binary search trees and hash tables. Make a six-question draft; each needs four options and a written explanation.

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

CODING 1 · INSERT
Create a seven-bucket string array, implement Hash(), and insert at least four test usernames. Print each chosen bucket index.

### Question 2

CODING 2 · COLLISION
Register 'ab' and 'ba'. Show that they collide and use linear probing to store both in different buckets.





NEW LEARNING

# Read, annotate and explain

### Implement hashing and collisions

Classroom demonstration only. Never type, collect or store real passwords.

### The problem: more than one user

A single username comparison is not scalable. We need a way to store and find many usernames.

### What is a hash function?

Convert the username to a number, then use % tableSize to choose a bucket index. Our small function is deliberately simple for learning.

### What if two names choose one bucket?

That is a collision. Use linear probing: try the next bucket, wrapping to the beginning when necessary.

### Lookups must use the same rule

Recalculate the starting bucket from the username, then probe through occupied buckets until the username or an empty bucket is found.

### Hash-table lookup versus password hashing

The hash-table function locates an account. It does NOT securely protect a password. Use invented demo passwords only.

### MCQ revision

Write four precise MCQs on linked lists, stacks, queues and trees; extend with BST and hash-table MCQs if time allows.

**Explain one step in your own words, using the correct technical terms.**

I DO — WORKED EXAMPLE

# Follow the worked method

## Build a real hash-table index

### Create the buckets

```
const int SIZE = 7;
string?[] usernames = new string?[SIZE];
```

### Write the index function

```
static int Hash(string key, int size)
{
    int total = 0;
    foreach (char letter in key)
        total += (int)letter;
    return total % size;
}
```

### Explain the limitation

```
// Hash("ab", 7) == Hash("ba", 7)
// Same total -> collision.
// This is an educational hash function,
// NOT a secure password hash.
```





WE DO — GUIDED PRACTICE

# Try → Discuss → Improve

**1. Insert with linear probing**

What happens when a bucket is already occupied?

Method / working:

**2. Search using the same probing sequence**

How can we find a name after a collision?

Method / working:

**3. Predict a collision**

With table size 7, why do 'ab' and 'ba' choose the same initial index?

Method / working: